

VoicelQ Commands Reference

Every command, script, or slash-command run during the VoicelQ Insight Service project session. Grouped by purpose. Each entry shows the exact form and a one-line explanation of what it does.

Environment + Shell setup

```
cd "E:\OneDrive\ICAFTWARE\Voice-iq(test1)\voiceiq-AI"
```

Change directory into the project root. Run before anything else.

```
.venv\Scripts\Activate.ps1
```

Activate the project Python virtual environment (Windows PowerShell). After this, `python` and `pip` refer to the venv copies, not the system ones.

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
```

Temporarily allow PowerShell to run local scripts in the current shell session only. Needed before activating the .venv on stricter Windows setups.

Git — daily workflow

```
git status
```

Show modified, staged, and untracked files. Run before every commit to confirm what will be committed.

```
git status --short
```

Compact one-line-per-file version of status. Easier to scan in long output.

```
git diff
```

Show unstaged line-level changes (what you've modified but not yet staged).

```
git diff --stat
```

Same as `git diff`, but summarized by file with insertion/deletion counts only. Useful when the full diff is huge.

```
git diff HEAD --stat
```

Show full diff against the most recent commit (includes both staged and unstaged).

```
git add <path>
```

Stage a specific file or directory for the next commit. Always prefer this over `git add .` so you never accidentally stage `.env` or `.venv`.

```
git add -u
```

Stage modifications + deletions of ALREADY-tracked files only. Won't sweep in new untracked files (e.g. a stray `.pyc`).

```
git commit -m "..."
```

Create a commit with a one-line message.

```
git commit -m "subject" -m "body"
```

Multi-paragraph commit: first -m is the subject line, every subsequent -m becomes a body paragraph.

```
git log --oneline -5
```

Show the last 5 commits, one per line.

```
git push origin main
```

Push the local main branch up to GitHub origin.

```
git pull origin main --no-rebase --no-edit
```

Pull and merge remote changes. `--no-edit` skips opening an editor for the merge commit message (use the default).

```
git pull origin main --allow-unrelated-histories
```

Pull when local and remote branches have no shared ancestor (e.g. just created repo + got out of sync).

Git — config + safety

```
git config --global core.editor notepad
```

Set Notepad as the git commit-message editor on Windows. Avoids vim swap-file disasters.

```
git config blame.ignoreRevsFile .git-blame-ignore-revs
```

Tell `git blame` to skip every SHA listed in `.git-blame-ignore-revs` (used to bypass the bulk-format commit so blame still shows real authors).

```
git ls-files | Select-String "\.env$"
```

PowerShell-friendly version of `git ls-files | grep .env`. Returns empty if `.env` is NOT tracked (good).

```
git remote -v
```

Show what URLs the local repo is pushing/pulling against.

```
git remote add origin https://github.com/HAMZAJAWED12/Voice-iq.git
```

Re-add the origin remote. Needed after `git filter-repo`, which strips remotes as a safety measure.

```
git rev-parse HEAD
```

Print the SHA of the current commit. Useful for piping into other commands or saving for later reference.

Git — fixing problems

```
git restore .
```

Discard ALL unstaged changes in the working tree. Returns every modified file to its last-committed state.

```
git restore <path>
```

Same, but only for one file.

```
git checkout --ours .
```

During a merge conflict: keep the LOCAL version of every conflicting file. Discards remote changes.

```
git merge --abort
```

Cancel an in-progress merge and return to the state before `git merge`/`git pull`. Use when you change your mind mid-conflict.

```
git stash / git stash pop
```

Temporarily set aside uncommitted changes, then restore them later. Useful when you need a clean tree to run a single command.

```
git rm -r --cached <path>
```

Untrack a file/directory but leave it on disk. Use when you accidentally committed something the .gitignore now covers (e.g. data/, __pycache__/).

```
Remove-Item .git\index.lock -Force
```

PowerShell: delete a stale git lock file. Needed when a previous git operation was killed mid-write (common on Windows + OneDrive).

```
git push origin main --force
```

Force-push, overwriting the remote branch with your local one. DESTRUCTIVE — anyone with a clone breaks until they re-clone. Used only after `git filter-repo`.

```
git filter-repo --path "app/insights.zip" --invert-paths --force
```

Rewrite every commit on every branch to pretend the named path never existed. Used to permanently purge a leaked secret from git history.

```
git log --all --pretty=format: --name-only --diff-filter=A | Select-String "\.zip$" |  
Sort-Object -Unique
```

Survey every .zip file ever added to any branch in history. Used to plan the filter-repo purge before running it.

Python + pip

```
pip install -r requirements-insight.txt
```

Install the lightweight insight-service dependencies (FastAPI, Pydantic, SQLAlchemy, pytest, httpx).

```
pip install pre-commit==4.0.1
```

Install the pre-commit framework, pinned to version 4.0.1.

```
pip install git-filter-repo
```

Install the modern history-rewriting tool. Replaces the deprecated `git filter-branch`.

```
python -m pytest -v app/insights/tests/
```

Run the insight-service test suite with verbose output.

```
python -m pytest -q app/insights/tests/
```

Quiet pytest run — shows only dots + final summary. Used for fast sanity checks after a refactor.

```
python -m ruff check . --statistics
```

Run ruff linter, print a one-line summary per error code (e.g. 'E501 42 line too long').

Pre-commit hooks

```
pre-commit install
```

Wire pre-commit into ``.git/hooks/pre-commit`` so it runs on every ``git commit``.

```
pre-commit run
```

Run hooks against currently STAGED files only (what's about to be committed).

```
pre-commit run --all-files
```

Run hooks against every tracked file. Used once after first install to baseline the codebase.

```
pre-commit autoupdate
```

Bump every pinned hook in `.pre-commit-config.yaml` to its latest release.

Application server

```
uvicorn app.main:app --host 0.0.0.0 --port 8888 --reload
```

Start the FastAPI service locally on port 8888 with hot-reload on source changes.

```
sqlite3 data/insights.db "SELECT id, conversation_id, claim_type, verdict FROM  
fact_check_results ORDER BY id DESC LIMIT 5;"
```

Query the persisted fact-check results in SQLite without writing Python code. Useful for quick verification after a Postman test.

API testing (curl / Postman)

```
curl http://localhost:8888/healthz
```

Hit the liveness probe. Expects 200 OK with `{"status": "ok"}`.

```
curl -X POST http://localhost:8888/v1/fact-check -H "Content-Type: application/json" -d  
'{"conversation_id":"c1","speaker_id":"s1","transcript_text":"Bitcoin is at 95000  
dollars"}'
```

Submit a transcript for fact-checking. Returns verdict, confidence, and evidence per detected claim.

```
curl http://localhost:8888/v1/fact-check/c1
```

Fetch all persisted fact-check results for a given `conversation_id`.

```
curl "https://api.openweathermap.org/data/2.5/weather?q=Karachi&appid=YOUR_OW_KEY"
```

Smoke test the OpenWeather API key. 200 + JSON = valid key. 401 = bad or unactivated key.

```
curl  
"https://www.alphavantage.co/query?function=GLOBAL_QUOTE&symbol=AAPL&apikey=YOUR_AV_KEY"
```

Smoke test the Alpha Vantage stock API key. Returns AAPL price as JSON.

```
curl "https://en.wikipedia.org/api/rest_v1/page/summary/France" -UserAgent "VoiceIQ"
```

Fetch the Wikipedia REST summary for a country page. Used to inspect what `extract` text our static-facts client must parse.

Docker

```
docker build -t voiceiq-insight:dev .
```

Build the production container image from the project Dockerfile, tagged `voiceiq-insight:dev`.

```
docker run --rm -d --name voiceiq-test -p 8888:8888 -v ${PWD}/data:/app/data -e  
VOICEIQ_DATABASE_AUTO_CREATE=true voiceiq-insight:dev
```

Run the image detached on port 8888, mounting host `data/` for SQLite persistence. `--rm` auto-deletes the container on exit.

```
docker exec voiceiq-test ls -la /app
```

Run a command inside the running container. Used to verify `.env` was NOT baked into the image.

```
docker stop voiceiq-test
```

Stop the named container. With `--rm` the container is also deleted.

```
docker compose up -d
```

Start every service in docker-compose.yml in the background. Mounts the named volume + .env file from the host.

```
docker compose down
```

Stop and remove every compose-managed service. Named volumes persist (your SQLite data survives).

```
docker tag voiceiq-insight:dev voiceiq-insight:1.0.0
```

Add a second tag to an existing image. Used to version-stamp a release.

Claude Code (npm install + launch)

```
npm install -g @anthropic-ai/claude-code
```

Install Claude Code globally via npm. Requires Node.js 18+.

```
claude --version
```

Verify the Claude Code install succeeded by printing its version.

```
claude
```

Launch Claude Code in the current directory. First run triggers OAuth login + folder-trust prompt.

PowerShell file utilities

```
Add-Content .gitignore "`n# Local Claude Code state`n.claude/"
```

Append two lines (header comment + path) to .gitignore. The backtick-n is PowerShell's newline escape.

```
Get-Content .env | Select-String "API_KEY"
```

Print every line in .env containing 'API_KEY'. Sanity-check after creating the file.

```
Remove-Item -Recurse -Force "E:\path\to\backup"
```

Delete a directory and everything inside it without prompting. Used to clean up the pre-purge backup once the purge succeeded.

```
Get-Process vim | Stop-Process -Force
```

Kill every running vim process. Used to clear a zombie editor that left a swap file behind.

```
Rename-Item run_eval_dev.py run_eval_dev.LOCAL.py
```

Rename a file. Used to step around a merge conflict on the original filename.

```
@"contents`n"@ | Out-File -Encoding utf8 -Append .env
```

PowerShell here-string. Appends a multi-line block to a file with UTF-8 encoding.

Skills + slash commands (inside Claude Code)

```
/init
```

Generate an initial CLAUDE.md from the current repo contents.

```
/review
```

Ask Claude Code to review the current branch or pending changes.

```
/security-review
```

Audit the working tree for secret leaks, injection risks, exception leaks, etc. Run before any push that touches auth or external API calls.

```
/help
```

Print the list of all built-in slash commands.

```
Shift+Tab
```

Toggle plan mode. Claude proposes a plan and waits for your approval before editing files.

```
Esc
```

Cancel the in-flight request.

```
Ctrl+R
```

Toggle verbose mode (shows every tool call inline).